Stacks and Queues

1. Write a program to implement stack from scratch in python using array.

```python
class Stack:
    def __init__(self, capacity):
        self.capacity = capacity
        # Pre-allocate the fixed-size array
        self.stack = [None] * capacity
        self.top = -1  # Points to the current top element

    def is_full(self):
        return self.top == self.capacity - 1

    def is_empty(self):
        return self.top == -1

    def push(self, item):
        if self.is_full():
            print("Stack Overflow")
            return
        self.top += 1
        self.stack[self.top] = item

    def pop(self):
        if self.is_empty():
            print("Stack Underflow")
            return None
        item = self.stack[self.top]
        self.stack[self.top] = None  # Optional: Clear reference
        self.top -= 1
        return item

    def peek(self):
        if self.is_empty():
            return None
        return self.stack[self.top]

    def size(self):
        return self.top + 1

# Example usage
if __name__ == "__main__":
    stack = Stack(5)

    stack.push(10)
    stack.push(20)
    stack.push(30)
```

```python
stack.push(40)

print("Peek:", stack.peek())  # 40
print("Pop:", stack.pop())    # 40
print("Size:", stack.size())  # 3

print("Peek after pop:", stack.peek())  # 30
```

Output:

```
Peek: 40
Pop: 40
Size: 3
Peek after pop: 30
```

2. Write a program in python to implement stack using python list.

```python
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack"""
        self.items.append(item)

    def pop(self):
        """Remove and return the top item from the stack"""
        if not self.is_empty():
            return self.items.pop()
        return None

    def peek(self):
        """Return the top item without removing it"""
        if not self.is_empty():
            return self.items[-1]
        return None

    def is_empty(self):
        """Check if the stack is empty"""
        return len(self.items) == 0
```

```python
    def size(self):
        """Return the number of items in the stack"""
        return len(self.items)

    def display(self):
        """Display all items in the stack"""
        print("Stack:", self.items)


# Example usage
if __name__ == "__main__":
    stack = Stack()

    # Push elements
    stack.push(10)
    stack.push(20)
    stack.push(30)
    stack.push(40)

    stack.display()  # Stack: [10, 20, 30, 40]

    print("Peek:", stack.peek())  # 40
    print("Pop:", stack.pop())    # 40
    print("Size:", stack.size())  # 3

    stack.display()  # Stack: [10, 20, 30]
```

Output:

```
Stack: [10, 20, 30, 40]
Peek: 40
Pop: 40
Size: 3
Stack: [10, 20, 30]
```

3. Write a program for Bracket Matching using using above created Stack.

```python
from .stack import Stack

def is_balanced(expression):
```

```
    st = Stack()
    pairs = {')': '(', '}': '{', ']': '['}
    for char in expression:
        if char in "({[":
            st.push(char)
        elif char in ")}]":
            if st.isEmpty() or st.pop() != pairs[char]:
                return False
    return st.isEmpty()

expr1 = "({[]})"
expr2 = "([)]"
print(expr1, "->", is_balanced(expr1))
print(expr2, "->", is_balanced(expr2))
```

Output:

```
({[]}) -> True
([)] -> False
```

4. Write a program in python to implement Infix to Postfix conversion using Stack implemented above.

```
from .stack import Stack

precedence = {'+': 1, '-': 1, '*': 2, '/': 2, '^': 3}

def infix_to_postfix(expression):
    st = Stack()
    output = []

    for token in expression:
        if token.isalnum():
            output.append(token)
        elif token == '(':
            st.push(token)
        elif token == ')':
            while not st.isEmpty() and st.peek() != '(':
                output.append(st.pop())
            st.pop()
        else:
            while (not st.isEmpty() and st.peek() != '(' and
                   precedence.get(st.peek(), 0) >= precedence.get(token, 0)):
                output.append(st.pop())
            st.push(token)
```

```python
    while not st.isEmpty():
        output.append(st.pop())

    return ''.join(output)

#Implementation

expr = "A+B*(C-D)"
print("Infix:", expr)
print("Postfix:", infix_to_postfix(expr))  # ABCD-*+
```

Output:

```
Infix: A+B*(C-D)
Postfix: ABCD-*+
```

5. Write a program in python to evluate postfix expression obtained from above implementation.

```python
from stack import Stack

def evaluate_postfix(expression):
    s = Stack()
    # Split by space if expression is like "5 3 +"
    # or iterate directly if it's "53+"
    for char in expression.split():
        if char.isdigit():
            s.push(int(char))
        else:
            val2 = s.pop()
            val1 = s.pop()

            if char == '+': s.push(val1 + val2)
            elif char == '-': s.push(val1 - val2)
            elif char == '*': s.push(val1 * val2)
            elif char == '/': s.push(val1 / val2)

    return s.pop()

if __name__ == "__main__":
    # Example: "10 2 + 3 *" -> (10 + 2) * 3 = 36
    expr = "10 2 + 3 *"
```

```
    print(f"{expr}: {evaluate_postfix(expr)}")
```

Output:

```
10 2 + 3 *: 36
```

6. Implement a queue using array in python.

```
class Queue: #circular
    def __init__(self, capacity):
        self.capacity = capacity
        self.queue = [None] * capacity
        self.front = 0
        self.rear = -1
        self.size = 0

    def is_full(self):
        return self.size == self.capacity

    def is_empty(self):
        return self.size == 0

    def enqueue(self, item):
        if self.is_full():
            print("Queue Overflow")
            return
        self.rear = (self.rear + 1) % self.capacity
        self.queue[self.rear] = item
        self.size += 1

    def dequeue(self):
        if self.is_empty():
            print("Queue Underflow")
            return None
        item = self.queue[self.front]
        self.queue[self.front] = None  # Optional: clear reference
        self.front = (self.front + 1) % self.capacity
        self.size -= 1
        return item

    def peek(self):
        if self.is_empty():
            return None
        return self.queue[self.front]
```

```python
#Example use

if __name__ == "__main__":
    q = Queue(5)

    q.enqueue(10)
    q.enqueue(20)
    q.enqueue(30)
    q.enqueue(40)
    q.enqueue(50)

    print("Front item:", q.peek())  # 10

    print("Dequeue:", q.dequeue()   # 10
    print("Dequeue:", q.dequeue()   # 20

    print("Front item after dequeue:", q.peek())  # 30

    q.enqueue(60)  # This will work as it's circular
    print("Front item after enqueueing 60:", q.peek())  # 30
```

Output:

```
Front item: 10
Dequeue: 10
Dequeue: 20
Front item after dequeue: 30
Front item after enqueueing 60: 30
```

7. Implement a priority queue using array in python.

```python
class PriorityQueue:
    def __init__(self, capacity):
        self.capacity = capacity
        # Pre-allocate the fixed-size "array"
        self.queue = [None] * capacity
        self.size = 0

    def is_full(self):
```

```python
        return self.size == self.capacity

    def is_empty(self):
        return self.size == 0

    def enqueue(self, data, priority):
        if self.is_full():
            print("Queue Overflow")
            return

        new_item = {'data': data, 'priority': priority}

        # If empty, just place at the start
        if self.is_empty():
            self.queue[0] = new_item
            self.size += 1
        else:
            # Start from the back and shift elements to the right
            # to make room for the new item (Sorted Insertion)
            i = self.size - 1
            while i >= 0 and self.queue[i]['priority'] < priority:
                self.queue[i + 1] = self.queue[i]
                i -= 1

            # Place the new item in its sorted position
            self.queue[i + 1] = new_item
            self.size += 1

    def dequeue(self):
        if self.is_empty():
            print("Queue Underflow")
            return None

        # In a sorted array, the highest priority is always at index 0
        highest = self.queue[0]

        # Shift everything left to fill the gap
        for i in range(self.size - 1):
            self.queue[i] = self.queue[i + 1]

        self.queue[self.size - 1] = None # Clean up
        self.size -= 1
        return highest

    def peek(self):
        if self.is_empty():
            return None
        return self.queue[0]
```

```python
# Example usage
if __name__ == "__main__":
    pq = PriorityQueue(5)

    pq.enqueue("Task 1", priority=2)
    pq.enqueue("Task 2", priority=5)
    pq.enqueue("Task 3", priority=1)

    print("Highest Priority Task:", pq.peek())  # Task 2

    print("Dequeue:", pq.dequeue())  # Task 2
    print("Next Highest Priority Task:", pq.peek())  # Task 1
```

Output:

```
Highest Priority Task: {'data': 'Task 2', 'priority': 5}
Dequeue: {'data': 'Task 2', 'priority': 5}
Next Highest Priority Task: {'data': 'Task 1', 'priority': 2}
```

Linked List

1. Write a program to show the implementation of Linked List in python.

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None  # Pointer to the next node

class LinkedList:
    def __init__(self):
        self.head = None  # The starting point of the list

    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return

        # Traverse to the end of the list
        last_node = self.head
        while last_node.next:
            last_node = last_node.next

        last_node.next = new_node

    def display(self):
        elements = []
        current = self.head
        while current:
            elements.append(str(current.data))
            current = current.next
        print(" -> ".join(elements) + " -> None")

# example usage
if __name__ == "__main__":
    ll = LinkedList()
    ll.append(10)
    ll.append(20)
    ll.append(30)

    ll.display()  # Output: 10 -> 20 -> 30 -> None
```

Output:

```
10 -> 20 -> 30 -> None
```

2. Write the implementation of Singly Linked Lists to show creation, traversal, insertion, retrieval and deletion of elements.

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node

    def insert_at_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def traverse(self):
        nodes = []
        temp = self.head
        while temp:
            nodes.append(str(temp.data))
            temp = temp.next
        print(" -> ".join(nodes) + " -> None")

    def search(self, key):
        temp = self.head
        index = 0
        while temp:
            if temp.data == key:
                return f"Found {key} at index {index}"
            temp = temp.next
            index += 1
        return "Value not found"

    def delete_node(self, key):
```

```python
        temp = self.head

        # Case 1: Head node holds the key
        if temp and temp.data == key:
            self.head = temp.next
            temp = None
            return

        # Case 2: Search for the key to be deleted
        prev = None
        while temp and temp.data != key:
            prev = temp
            temp = temp.next

        # Case 3: Key not found
        if temp is None:
            return

        # Unlink the node from linked list
        prev.next = temp.next
        temp = None

# Example usage
if __name__ == "__main__":
    ll = LinkedList()
    ll.insert_at_end(10)
    ll.insert_at_end(20)
    ll.insert_at_beginning(5)
    ll.traverse()  # Output: 5 -> 10 -> 20 -> None
    print(ll.search(10))  # Output: Found 10 at index 1
    print(ll.search(15))  # Output: Value not found
    ll.delete_node(10)
    ll.traverse()  # Output: 5 -> 20 -> None
```

Output:

```
5 -> 10 -> 20 -> None
Found 10 at index 1
Value not found
5 -> 20 -> None
```

3. Write a program to implement the Doubly Linked List.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_beginning(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return

        new_node.next = self.head
        self.head.prev = new_node
        self.head = new_node

    def insert_at_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return

        temp = self.head
        while temp.next:
            temp = temp.next

        temp.next = new_node
        new_node.prev = temp

    def delete_node(self, key):
        temp = self.head

        # Search for the node to delete
        while temp and temp.data != key:
            temp = temp.next

        if temp is None: # Key not found
            return

        # If the node to be deleted is the head node
```

```python
        if temp == self.head:
            self.head = temp.next

        # Change next only if node to be deleted is NOT the last node
        if temp.next is not None:
            temp.next.prev = temp.prev

        # Change prev only if node to be deleted is NOT the first node
        if temp.prev is not None:
            temp.prev.next = temp.next

        temp = None # Free memory

    def display_forward(self):
        temp = self.head
        elements = []
        while temp:
            elements.append(str(temp.data))
            temp = temp.next
        print("Forward:  " + " <-> ".join(elements) + " -> None")

    def display_backward(self):
        temp = self.head
        if not temp: return

        # Move to the end of the list
        while temp.next:
            temp = temp.next

        # Traverse backward using 'prev'
        elements = []
        while temp:
            elements.append(str(temp.data))
            temp = temp.prev
        print("Backward: " + " <-> ".join(elements) + " -> None")

# Example usage
if __name__ == "__main__":
    dll = DoublyLinkedList()
    dll.insert_at_end(10)
    dll.insert_at_end(20)
    dll.insert_at_beginning(5)
    dll.insert_at_beginning(1)

    dll.display_forward()  # Output: Forward:  1 <-> 5 <-> 10 <-> 20 -> None
    dll.display_backward() # Output: Backward: 20 <-> 10 <-> 5 <-> 1 -> None

    print(dll.delete_node(10)) # Deletes node with value 10
```

```
    dll.display_forward()  # Output: Forward:  1 <-> 5 <-> 20 -> None
```

Output:

```
Forward:  1 <-> 5 <-> 10 <-> 20 -> None
Backward: 20 <-> 10 <-> 5 <-> 1 -> None
None
Forward:  1 <-> 5 <-> 20 -> None
```

4. Write a program to implement Circular Lists.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularLinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            new_node.next = self.head
            return

        temp = self.head
        # Traverse to the last node
        while temp.next != self.head:
            temp = temp.next

        temp.next = new_node
        new_node.next = self.head

    def traverse(self):
        if not self.head:
            print("List is empty")
            return

        temp = self.head
        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
```

```
            if temp == self.head:
                break
        print("(Back to Head)")

    def delete(self, key):
        if self.head is None:
            return

        curr = self.head
        prev = None

        # Case 1: Deleting the Head
        if curr.data == key:
            # If there's only one node
            if curr.next == self.head:
                self.head = None
                return

            # Find the last node to update its pointer
            last = self.head
            while last.next != self.head:
                last = last.next

            self.head = self.head.next
            last.next = self.head
            return

        # Case 2: Deleting middle or last node
        while curr.next != self.head:
            prev = curr
            curr = curr.next
            if curr.data == key:
                prev.next = curr.next
                return

# example usage
if __name__ == "__main__":
    cll = CircularLinkedList()
    cll.insert(10)
    cll.insert(20)
    cll.insert(30)
    cll.insert(40)

    print("Circular Linked List after insertion:")
    cll.traverse()

    cll.delete(20)
    print("\nCircular Linked List after deleting 20:")
```

```python
    cll.traverse()

    cll.delete(10)
    print("\nCircular Linked List after deleting 10 (head):")
    cll.traverse()
```

Output:

```
10 -> 20 -> 30 -> 40 -> (Back to Head)

Circular Linked List after deleting 20:
10 -> 30 -> 40 -> (Back to Head)

Circular Linked List after deleting 10 (head):
30 -> 40 -> (Back to Head)
```

5. Show the implementation of stack and queue using the Linked List created above.

STACK:

```python
# STACK USING SINGLY LINKED LIST

from .singly_ll import SinglyLinkedList

class Stack:
    def __init__(self):
        # Internal storage using your Singly Linked List
        self._storage = SinglyLinkedList()
        self.size = 0

    def push(self, data):
        # Push is O(1) if we insert at the beginning (head)
        self._storage.insert_at_beginning(data)
        self.size += 1
        print(f"Pushed {data} to stack.")

    def pop(self):
        if self.is_empty():
            return "Stack Underflow"
```

```python
        # Get data from current head
        popped_node_data = self._storage.head.data
        # Move head pointer to the next node (effectively deleting the top)
        self._storage.head = self._storage.head.next
        self.size -= 1
        return popped_node_data

    def peek(self):
        return self._storage.head.data if self._storage.head else "Stack is empty"

    def is_empty(self):
        return self._storage.head is None

    def display_stack(self):
        print("Stack (Top -> Bottom):")
        self._storage.traverse()

# Example usage:
if __name__ == "__main__":
    stack = Stack()
    stack.push(10)
    stack.push(20)
    stack.push(30)
    stack.display_stack()
    print(f"Popped: {stack.pop()}")
    print(f"Top element: {stack.peek()}")
```

Output:

```
Pushed 10 to stack.
Pushed 20 to stack.
Pushed 30 to stack.
Stack (Top -> Bottom):
30 -> 20 -> 10 -> None
Popped: 30
Top element: 20
```

QUEUE:

```python
from .doubly_ll import DoublyLinkedList

class Queue:
    def __init__(self):
        # Internal storage using your Doubly Linked List
        self._storage = DoublyLinkedList()
        self.size = 0

    def enqueue(self, data):
        # Add to the end of the list
        self._storage.insert_at_end(data)
        self.size += 1
        print(f"Enqueued {data} to queue.")

    def dequeue(self):
        if self.is_empty():
            return "Queue Underflow"

        # In FIFO, we remove from the front (head)
        dequeued_data = self._storage.head.data
        # Use your existing DLL delete method
        self._storage.delete_node(dequeued_data)
        self.size -= 1
        return dequeued_data

    def is_empty(self):
        return self._storage.head is None

    def display_queue(self):
        print("Queue (Front -> Rear):")
        self._storage.display_forward()

# example usage:
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(10)
    queue.enqueue(20)
    queue.enqueue(30)
    queue.display_queue()
    print(f"Dequeued: {queue.dequeue()}")
    queue.display_queue()
```

Output:

```
Enqueued 10 to queue.
Enqueued 20 to queue.
Enqueued 30 to queue.
Queue (Front -> Rear):
Forward:  10 <-> 20 <-> 30 -> None
Dequeued: 10
Queue (Front -> Rear):
Forward:  20 <-> 30 -> None
```

Trees

1. Write a program to implement tree data structure in python. User must be able to insert and retrieve data from tree.

```python
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.children = []
        self.parent = None

    def add_child(self, child_node):
        child_node.parent = self
        self.children.append(child_node)

    def get_level(self):
        level = 0
        p = self.parent
        while p:
            level += 1
            p = p.parent
        return level

    def print_tree(self):
        spaces = ' ' * self.get_level() * 3
        prefix = spaces + "|__ " if self.parent else ""
        print(prefix + self.data)
        if self.children:
            for child in self.children:
                child.print_tree()

# --- Example Usage ---

# 1. Create the root
root = TreeNode("Electronics")

# 2. Create categories (Children of root)
laptop = TreeNode("Laptops")
phone = TreeNode("Cell Phones")

root.add_child(laptop)
root.add_child(phone)

# 3. Create specific items (Children of categories)
```

```python
laptop.add_child(TreeNode("MacBook Pro"))
laptop.add_child(TreeNode("Surface Pro"))
phone.add_child(TreeNode("iPhone 15"))

# 4. Retrieve/Display the structure
root.print_tree()
```

Output:

```
Electronics
    |__ Laptops
        |__ MacBook Pro
        |__ Surface Pro
    |__ Cell Phones
        |__ iPhone 15
```

2. Implement BFS for value retrieval from tree in python.

```python
from collections import deque

class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

def bfs_search(root, target):
    if root is None:
        return None

    # Initialize queue with the root node
    queue = deque([root])

    while queue:
        # Remove the first node in the queue
        current_node = queue.popleft()

        # Check if this is the value we want
```

```python
        if current_node.value == target:
            return current_node

        # Add children to the queue for the next level's exploration
        if current_node.left:
            queue.append(current_node.left)
        if current_node.right:
            queue.append(current_node.right)

    return None # Value not found

# Example Usage:
#      10
#     /  \
#    5    15
#   / \
#  2   7

root = Node(10)
root.left = Node(5)
root.right = Node(15)
root.left.left = Node(2)
root.left.right = Node(7)

result = bfs_search(root, 7)
print(f"Found node with value: {result.value}" if result else "Value not found")
```

Output:

```
Found node with value: 7
```

3. Implement DFS for value retrieval from tree in python.

```python
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

def dfs_search(node, target):
    # Base case: we hit a leaf's child or the tree is empty
    if node is None:
```

```
        return None

    # Check if this node is the one we want
    if node.value == target:
        return node

    # Search the left subtree
    found_left = dfs_search(node.left, target)
    if found_left:
        return found_left

    # If not in left, search the right subtree
    return dfs_search(node.right, target)

# Example Tree Setup
#      A(10)
#     /    \
#   B(5)   C(15)
#   /
# D(2)

root = Node(10)
root.left = Node(5)
root.right = Node(15)
root.left.left = Node(2)

result = dfs_search(root, 15)
print(f"Node found: {result.value}" if result else "Not found")
```

Output:

```
Node found: 15
```

4. Write a program in python to implement Binary Search tree.

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

class BinarySearchTree:
    def __init__(self):
```

```python
        self.root = None

    def insert(self, key):
        if self.root is None:
            self.root = Node(key)
        else:
            self._insert_recursive(self.root, key)

    def _insert_recursive(self, current, key):
        if key < current.key:
            if current.left is None:
                current.left = Node(key)
            else:
                self._insert_recursive(current.left, key)
        elif key > current.key:
            if current.right is None:
                current.right = Node(key)
            else:
                self._insert_recursive(current.right, key)
        # If key == current.key, we do nothing (no duplicates)

    def search(self, key):
        return self._search_recursive(self.root, key)

    def _search_recursive(self, current, key):
        # Base Case: Not found or found the node
        if current is None or current.key == key:
            return current

        # Value is greater than node's key
        if key > current.key:
            return self._search_recursive(current.right, key)

        # Value is smaller than node's key
        return self._search_recursive(current.left, key)

# --- Example Usage ---
bst = BinarySearchTree()
elements = [50, 30, 70, 20, 40, 60, 80]

for el in elements:
    bst.insert(el)

# Search for a value
target = 60
result = bst.search(target)

if result:
```

```
    print(f"Found {target} in the BST!")
else:
    print(f"{target} not found.")
```

Output:

```
Found 60 in the BST!
```

5. Implement AVL Tree in python.

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1

class AVLTree:
    def get_height(self, node):
        return node.height if node else 0

    def get_balance(self, node):
        return self.get_height(node.left) - self.get_height(node.right) if node else 0

    def rotate_right(self, y):
        x = y.left
        T2 = x.right
        # Perform rotation
        x.right = y
        y.left = T2
        # Update heights
        y.height = 1 + max(self.get_height(y.left), self.get_height(y.right))
        x.height = 1 + max(self.get_height(x.left), self.get_height(x.right))
        return x

    def rotate_left(self, x):
        y = x.right
        T2 = y.left
        # Perform rotation
        y.left = x
        x.right = T2
        # Update heights
        x.height = 1 + max(self.get_height(x.left), self.get_height(x.right))
```

```python
        y.height = 1 + max(self.get_height(y.left), self.get_height(y.right))
        return y

    def insert(self, root, key):
        # 1. Standard BST insertion
        if not root:
            return Node(key)
        elif key < root.key:
            root.left = self.insert(root.left, key)
        else:
            root.right = self.insert(root.right, key)

        # 2. Update height of ancestor node
        root.height = 1 + max(self.get_height(root.left), self.get_height(root.right))

        # 3. Get balance factor to check if it became unbalanced
        balance = self.get_balance(root)

        # 4. If unbalanced, try the 4 cases:
        # Left Left Case
        if balance > 1 and key < root.left.key:
            return self.rotate_right(root)

        # Right Right Case
        if balance < -1 and key > root.right.key:
            return self.rotate_left(root)

        # Left Right Case
        if balance > 1 and key > root.left.key:
            root.left = self.rotate_left(root.left)
            return self.rotate_right(root)

        # Right Left Case
        if balance < -1 and key < root.right.key:
            root.right = self.rotate_right(root.right)
            return self.rotate_left(root)

        return root

    def pre_order(self, root):
        if not root:
            return
        print(f"{root.key} ", end="")
        self.pre_order(root.left)
        self.pre_order(root.right)

# --- Example Usage ---
tree = AVLTree()
```

```python
root = None
nums = [10, 20, 30, 40, 50, 25]

for num in nums:
    root = tree.insert(root, num)

print("Pre-order traversal of the balanced AVL tree:")
tree.pre_order(root)
```

Output:

```
Pre-order traversal of the balanced AVL tree:
30 20 10 25 40 50 %
```

6. Write a program to implement Heap data structure in python.

```python
import heapq

class MinHeap:
    def __init__(self):
        # Initialize an empty list to represent the heap
        self.heap = []

    def push(self, val):
        """Adds an element to the heap while maintaining the heap property."""
        heapq.heappush(self.heap, val)
        print(f"Pushed {val}: {self.heap}")

    def pop(self):
        """Removes and returns the smallest element (the root)."""
        if self.heap:
            removed = heapq.heappop(self.heap)
            print(f"Popped {removed}: {self.heap}")
            return removed
        return "Heap is empty"

    def peek(self):
        """Returns the smallest element without removing it."""
        return self.heap[0] if self.heap else None

    def size(self):
        return len(self.heap)

# --- Example Usage ---
```

```python
my_heap = MinHeap()

# Adding elements
my_heap.push(10)
my_heap.push(5)
my_heap.push(15)
my_heap.push(1)

print(f"\nSmallest element (peek): {my_heap.peek()}")

# Removing elements (will always be the smallest)
my_heap.pop()
my_heap.pop()
```

Output:

```
Pushed 10: [10]
Pushed 5: [5, 10]
Pushed 15: [5, 10, 15]
Pushed 1: [1, 5, 15, 10]

Smallest element (peek): 1
Popped 1: [5, 10, 15]
Popped 5: [10, 15]
```

7. Write a program to implement priority queue using heap data structure in python.

```python
import heapq

class PriorityQueue:
    def __init__(self):
        self._queue = []
        self._index = 0  # Helps maintain order for items with identical priority

    def push(self, item, priority):
        """
        Inserts an item with a given priority.
```

```python
        We include self._index to handle cases where two items have the
        same priority, ensuring they are returned in the order they were added.
        """
        heapq.heappush(self._queue, (priority, self._index, item))
        self._index += 1
        print(f"Added '{item}' with priority {priority}")

    def pop(self):
        """Removes and returns the item with the highest priority (lowest value)."""
        if not self._queue:
            return "Queue is empty"

        priority, index, item = heapq.heappop(self._queue)
        return f"Processing: {item} (Priority: {priority})"

    def is_empty(self):
        return len(self._queue) == 0

# --- Example Usage ---
pq = PriorityQueue()

pq.push("Fix critical server bug", 1)   # High priority
pq.push("Check emails", 3)              # Low priority
pq.push("Write documentation", 2)       # Medium priority
pq.push("Reply to Slack message", 2)    # Medium priority (added after docs)

print("\n--- Processing Tasks ---")
while not pq.is_empty():
    print(pq.pop())
```

Output:

```
Added 'Fix critical server bug' with priority 1
Added 'Check emails' with priority 3
Added 'Write documentation' with priority 2
Added 'Reply to Slack message' with priority 2

--- Processing Tasks ---
Processing: Fix critical server bug (Priority: 1)
Processing: Write documentation (Priority: 2)
Processing: Reply to Slack message (Priority: 2)
Processing: Check emails (Priority: 3)
```

Searching

1. Write a program to show the implementation of sequential Search.

```
def sequential_search(data_list, target):
    for index in range(len(data_list)):
        if data_list[index] == target:
            return index  # Match found, return the position

    return -1  # End of list reached without finding the target

# Example Usage:
numbers = [10, 23, 45, 70, 11, 15]
search_item = 70

result = sequential_search(numbers, search_item)

if result != -1:
    print(f"Element {search_item} found at index {result}.")
else:
    print(f"Element {search_item} not found in the list.")
```

Output:

```
Element 70 found at index 3.
```

2. Write the implementation of Binary Search in Python.

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        # Find the middle index
        mid = (low + high) // 2

        # Check if the target is at the midpoint
        if arr[mid] == target:
            return mid

        # If target is greater, ignore the left half
        elif arr[mid] < target:
            low = mid + 1
```

```python
        # If target is smaller, ignore the right half
        else:
            high = mid - 1

    # Target was not present in the list
    return -1

# Example Usage:
my_list = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
target_val = 23

result = binary_search(my_list, target_val)

if result != -1:
    print(f"Element found at index {result}")
else:
    print("Element not found")
```

Output:

```
Element found at index 5
```

3. Show the basic implementation of Hashing in python

```python
class SimpleHashTable:
    def __init__(self, size):
        self.size = size
        self.table = [None] * size

    def _hash_function(self, key):
        """Generates an index for a given string key."""
        # Sum the ASCII values of the characters and take the modulo
        return sum(ord(char) for char in str(key)) % self.size

    def insert(self, key, value):
        """Inserts a key-value pair into the table."""
        index = self._hash_function(key)
        self.table[index] = value
        print(f"Inserted '{value}' at index {index}")

    def get(self, key):
        """Retrieves a value based on the key."""
        index = self._hash_function(key)
        return self.table[index]
```

```python
# --- Testing the Implementation ---
my_hash = SimpleHashTable(10)

my_hash.insert("name", "Alice")
my_hash.insert("id", 4021)

print(f"Value for 'name': {my_hash.get('name')}")
```

Output:

```
Inserted 'Alice' at index 7
Inserted '4021' at index 5
Value for 'name': Alice
```

4. Write a program to show linear, quadratic probing and double hashing in python.

```python
class OpenAddressingHashTable:
    def __init__(self, size):
        self.size = size
        self.table = [None] * size

    def hash1(self, key):
        return key % self.size

    def hash2(self, key):
        # Secondary hash for Double Hashing (must not return 0)
        return 7 - (key % 7)

    def insert_linear(self, key):
        index = self.hash1(key)
        i = 0
        while self.table[(index + i) % self.size] is not None:
            i += 1
        self.table[(index + i) % self.size] = key

    def insert_quadratic(self, key):
        index = self.hash1(key)
        i = 0
        while self.table[(index + i**2) % self.size] is not None:
```

```
            i += 1
        self.table[(index + i**2) % self.size] = key

    def insert_double_hash(self, key):
        index = self.hash1(key)
        step = self.hash2(key)
        i = 0
        while self.table[(index + i * step) % self.size] is not None:
            i += 1
        self.table[(index + i * step) % self.size] = key

# --- Comparison Test ---
keys = [12, 22, 32] # All these will collide if size is 10
size = 10

# 1. Linear Probing
lp = OpenAddressingHashTable(size)
for k in keys: lp.insert_linear(k)
print(f"Linear Probing:    {lp.table}")

# 2. Quadratic Probing
qp = OpenAddressingHashTable(size)
for k in keys: qp.insert_quadratic(k)
print(f"Quadratic Probing: {qp.table}")

# 3. Double Hashing
dh = OpenAddressingHashTable(size)
for k in keys: dh.insert_double_hash(k)
print(f"Double Hashing:    {dh.table}")
```

Output:

```
Linear Probing:    [None, None, 12, 22, 32, None, None, None, None, None]
Quadratic Probing: [None, None, 12, 22, None, None, 32, None, None, None]
Double Hashing:    [None, None, 12, None, None, 32, None, None, 22, None]
```

5. Write a program to implement Separate Chaining in python.

```
class ChainedHashTable:
    def __init__(self, size):
        self.size = size
        # Create an array where each element is an empty list (the "chain")
        self.table = [[] for _ in range(self.size)]

    def _hash_function(self, key):
```

```python
        return hash(key) % self.size

    def insert(self, key, value):
        index = self._hash_function(key)

        # Check if the key already exists in the chain to update it
        for i, kv_pair in enumerate(self.table[index]):
            if kv_pair[0] == key:
                self.table[index][i] = (key, value)
                return

        # Otherwise, append the new key-value pair to the list
        self.table[index].append((key, value))

    def search(self, key):
        index = self._hash_function(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return None

    def display(self):
        for i, chain in enumerate(self.table):
            print(f"Bucket {i}: {chain}")

# --- Testing ---
ht = ChainedHashTable(5)
ht.insert("Apple", 100)
ht.insert("Banana", 200)
ht.insert("Cherry", 300) # Might collide depending on hash
ht.insert("Date", 400)

ht.display()
print(f"\nSearching for 'Banana': {ht.search('Banana')}")
```

Output:

```
Bucket 0: []
Bucket 1: [('Apple', 100), ('Cherry', 300)]
Bucket 2: [('Banana', 200)]
Bucket 3: []
Bucket 4: [('Date', 400)]

Searching for 'Banana': 200
```

Sorting

1. Write a program to implement

a) Bubble Sort

```
def bubble_sort(arr):
    n = len(arr)

    # Traverse through all array elements
    for i in range(n):
        # Flag to optimize: if no two elements were swapped by inner loop, then break
        swapped = False

        # Last i elements are already in place
        for j in range(0, n - i - 1):

            # Traverse the array from 0 to n-i-1
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # If no elements were swapped, the list is already sorted
        if not swapped:
            break

# Example usage:
data = [64, 34, 25, 12, 22, 11, 90]
bubble_sort(data)
print("Sorted array:", data)
```

Output:

```
Sorted array: [11, 12, 22, 25, 34, 64, 90]
```

b) Insertion Sort

```
def insertion_sort(arr):
    # Start from the second element (index 1) as the first is "sorted"
    for i in range(1, len(arr)):
        key = arr[i]

        # Move elements of arr[0..i-1] that are greater than key
```

```python
        # to one position ahead of their current position
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1

        # Place the key in its correct location
        arr[j + 1] = key

# Example usage:
data = [12, 11, 13, 5, 6]
insertion_sort(data)
print("Sorted array:", data)
```

Output:

```
Sorted array: [5, 6, 11, 12, 13]
```

c) Selection Sort

```python
def selection_sort(arr):
    n = len(arr)

    # One by one move boundary of unsorted subarray
    for i in range(n):
        # Find the minimum element in remaining unsorted array
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j

        # Swap the found minimum element with the first element
        arr[i], arr[min_idx] = arr[min_idx], arr[i]

# Example usage:
data = [64, 25, 12, 22, 11]
selection_sort(data)
print("Sorted array:", data)
```

Output:

```
Sorted array: [11, 12, 22, 25, 64]
```

d) Merge Sort

```python
def merge_sort(arr):
    if len(arr) > 1:
        # Finding the mid of the array
        mid = len(arr) // 2

        # Dividing the array elements into 2 halves
        L = arr[:mid]
        R = arr[mid:]

        # Sorting the first and second halves
        merge_sort(L)
        merge_sort(R)

        i = j = k = 0

        # Copy data to temp arrays L[] and R[]
        while i < len(L) and j < len(R):
            if L[i] <= R[j]:
                arr[k] = L[i]
                i += 1
            else:
                arr[k] = R[j]
                j += 1
            k += 1

        # Checking if any element was left in L
        while i < len(L):
            arr[k] = L[i]
            i += 1
            k += 1

        # Checking if any element was left in R
        while j < len(R):
            arr[k] = R[j]
            j += 1
            k += 1

# Example usage:
data = [38, 27, 43, 3, 9, 82, 10]
merge_sort(data)
print("Sorted array:", data)
```

Output:

```
Sorted array: [3, 9, 10, 27, 38, 43, 82]
```

e) Quick Sort

```python
def partition(arr, low, high):
    # Choosing the rightmost element as pivot
    pivot = arr[high]

    # Index of smaller element
    i = low - 1

    for j in range(low, high):
        # If current element is smaller than or equal to pivot
        if arr[j] <= pivot:
            i += 1
            # Swap elements
            arr[i], arr[j] = arr[j], arr[i]

    # Swap the pivot element with the element at i+1
    arr[i + 1], arr[high] = arr[high], arr[i + 1]

    # Return the position where partition is done
    return i + 1

def quick_sort(arr, low, high):
    if low < high:
        # pi is partitioning index, arr[pi] is now at right place
        pi = partition(arr, low, high)

        # Separately sort elements before partition and after partition
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

# Example usage:
data = [10, 7, 8, 9, 1, 5]
quick_sort(data, 0, len(data) - 1)
print("Sorted array:", data)
```

Output:

```
Sorted array: [1, 5, 7, 8, 9, 10]
```

f) Heap sort

```python
import heapq

def heap_sort_simple(arr):
    # 1. Create a min-heap from the array
```

```python
    heapq.heapify(arr)

    # 2. Pop elements one by one to get them in sorted order
    # Note: This returns a new list; heapq.heapify modifies the list in-place
    return [heapq.heappop(arr) for _ in range(len(arr))]

# Alternative: In-place sort (ascending)
def heap_sort_inplace(arr):
    # heapq doesn't have a direct "heap_sort" function,
    # but we can simulate it:
    h = []
    for value in arr:
        heapq.heappush(h, value)

    for i in range(len(arr)):
        arr[i] = heapq.heappop(h)

# Example usage:
data = [12, 11, 13, 5, 6, 7]
sorted_data = heap_sort_simple(data)
print("Sorted array:", sorted_data)
```

Output:

```
Sorted array: [5, 6, 7, 11, 12, 13]
```

Graphs

1. Write a program to implement graph traversal using BFS and DFS.

```python
from collections import deque

class Graph:
    def __init__(self):
        # Using a dictionary to store the adjacency list
        self.graph = {}

    def add_edge(self, u, v):
        if u not in self.graph:
            self.graph[u] = []
        self.graph[u].append(v)
        # For undirected graphs, uncomment the next line:
        # if v not in self.graph: self.graph[v] = []
        # self.graph[v].append(u)

    def bfs(self, start_node):
        visited = set()
        queue = deque([start_node])
        visited.add(start_node)

        result = []
        while queue:
            vertex = queue.popleft()
            result.append(vertex)

            for neighbor in self.graph.get(vertex, []):
                if neighbor not in visited:
                    visited.add(neighbor)
                    queue.append(neighbor)
        return result

    def dfs(self, start_node, visited=None):
        if visited is None:
            visited = set()

        visited.add(start_node)
        traversal = [start_node]

        for neighbor in self.graph.get(start_node, []):
            if neighbor not in visited:
                traversal.extend(self.dfs(neighbor, visited))
        return traversal

# --- Example Usage ---
```

```python
g = Graph()
g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(1, 2)
g.add_edge(2, 0)
g.add_edge(2, 3)
g.add_edge(3, 3)

start = 2
print(f"BFS starting from node {start}: {g.bfs(start)}")
print(f"DFS starting from node {start}: {g.dfs(start)}")
```

Output:

```
BFS starting from node 2: [2, 0, 3, 1]
DFS starting from node 2: [2, 0, 1, 3]
```

2. Write a program to implement Kruskal's Algorithm for minimum spanning tree.

```python
class DisjointSet:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, i):
        if self.parent[i] == i:
            return i
        # Path compression
        self.parent[i] = self.find(self.parent[i])
        return self.parent[i]

    def union(self, i, j):
        root_i = self.find(i)
        root_j = self.find(j)

        if root_i != root_j:
            # Union by rank
            if self.rank[root_i] < self.rank[root_j]:
                self.parent[root_i] = root_j
            elif self.rank[root_i] > self.rank[root_j]:
                self.parent[root_j] = root_i
            else:
                self.parent[root_i] = root_j
                self.rank[root_j] += 1
```

```python
            return True
        return False

def kruskal_mst(vertices, edges):
    # Sort edges based on weight: (u, v, weight)
    edges.sort(key=lambda x: x[2])

    ds = DisjointSet(vertices)
    mst = []
    mst_weight = 0

    for u, v, weight in edges:
        # If u and v are in different sets, adding this edge won't form a cycle
        if ds.union(u, v):
            mst.append((u, v, weight))
            mst_weight += weight

    return mst, mst_weight

# --- Example Usage ---
# Graph represented as (u, v, weight)
num_vertices = 4
graph_edges = [
    (0, 1, 10),
    (0, 2, 6),
    (0, 3, 5),
    (1, 3, 15),
    (2, 3, 4)
]

mst_result, total_weight = kruskal_mst(num_vertices, graph_edges)

print("Edges in the Constructed MST:")
for u, v, weight in mst_result:
    print(f"{u} -- {v} == {weight}")
print(f"Minimum Spanning Tree Weight: {total_weight}")
```

Output:

```
Edges in the Constructed MST:
2 -- 3 == 4
0 -- 3 == 5
0 -- 1 == 10
Minimum Spanning Tree Weight: 19
```

3. Write a program to implement Prim's Algorithm to illustrate minimum spanning tree.

```python
import heapq

def prims_mst(vertices, adj_list):
    # (weight, node, parent)
    min_heap = [(0, 0, -1)]
    visited = [False] * vertices
    mst = []
    total_weight = 0

    while min_heap:
        weight, u, parent = heapq.heappop(min_heap)

        # If node is already in MST, skip it
        if visited[u]:
            continue

        # Add node to MST
        visited[u] = True
        total_weight += weight
        if parent != -1:
            mst.append((parent, u, weight))

        # Check neighbors
        for v, edge_weight in adj_list[u]:
            if not visited[v]:
                heapq.heappush(min_heap, (edge_weight, v, u))

    return mst, total_weight

# --- Example Usage ---
# Adjacency List: { node: [(neighbor, weight), ...] }
num_vertices = 5
adj = {
    0: [(1, 2), (3, 6)],
    1: [(0, 2), (2, 3), (3, 8), (4, 5)],
    2: [(1, 3), (4, 7)],
    3: [(0, 6), (1, 8)],
    4: [(1, 5), (2, 7)]
}

mst_result, weight_sum = prims_mst(num_vertices, adj)

print("Edges in the Prim's MST:")
for u, v, w in mst_result:
    print(f"{u} - {v} (Weight: {w})")
print(f"Total MST Weight: {weight_sum}")
```

Output:

```
Edges in the Prim's MST:
0 - 1 (Weight: 2)
1 - 2 (Weight: 3)
1 - 4 (Weight: 5)
0 - 3 (Weight: 6)
Total MST Weight: 16
```

Algorithm design

1. Write a program to show the implementation of dynamic programming in fibonacci series
Implementation

```python
def fibonacci_dp(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1

    # We only need to keep track of the last two numbers
    prev_2 = 0  # F(n-2)
    prev_1 = 1  # F(n-1)
    current = 0

    for i in range(2, n + 1):
        current = prev_1 + prev_2
        # Update our markers for the next iteration
        prev_2 = prev_1
        prev_1 = current

    return current

# Example Usage:
n_terms = 10
print(f"The {n_terms}th Fibonacci number is: {fibonacci_dp(n_terms)}")
```

Output:

```
The 10th Fibonacci number is: 55
```